Modified Newton methods with matrix sketching for large-scale smooth optimization

Junhyung Chang
New York University
jc10314@nyu.edu

Abstract

Under desirable conditions, Newton's method shows local quadratic convergence for smooth and strongly convex functions of the form $f : \mathbb{R}^d \to \mathbb{R}$. However, there are several cases where the pure Newton's method does not converge to a local minimizer, which makes it difficult to use in many practical settings. This project attempts to investigate the possible modifications that fix these issues, while retaining superlinear convergence that comes with using second-order information. The topics discussed in this project include line-search methods for global convergence on convex functions, and methods of modifying the Hessian matrix when it is non-positive definite, or ill-conditioned. Finally, there is a brief introduction on randomized methods known as matrix sketching, and how such methods can be applied to extend Newton's method to minimize a large sum of loss functions, which is a central topic in data science.

Contents

1 Introduction **2**

2 Modified Newton techniques **3**
2.1 Dealing with non-convex functions or ill-conditioned Hessian 3
2.2 Numerical experiment: ordinary least squares regression 4

3 Matrix sketching techniques **7**
3.1 Overview of methods 8
i Newton Sketch 8
ii Row-sampled Newton 9
iii Stochastic Gradient Descent 9
3.2 Numerical experiments 9
3.3 Remarks 10

4 Conclusion **11**

1. Introduction

At each point where the function $f : \mathbb{R}^d \to \mathbb{R}$ is twice continuously differentiable, f can be approximated by a local quadratic model based on the gradient and Hessian of f:

$$q(\mathbf{x}) = f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^T(\mathbf{x} - \mathbf{x}_k) + \frac{1}{2}(\mathbf{x} - \mathbf{x}_k)^T \nabla^2 f(\mathbf{x}_k)(\mathbf{x} - \mathbf{x}_k), \tag{1.1}$$

with error

$$|f(\mathbf{x}) - q(\mathbf{x})| = \mathcal{O}(|\mathbf{x} - \mathbf{x}_k|^3). \tag{1.2}$$

If f is strongly convex at x_k, Newton's method produces x_{k+1} by finding the minimum of the local quadratic model of f at the current location x_k. This is done by finding the point where the gradient of the local quadratic model is zero:

$$\nabla q(\mathbf{x}) = \nabla f(\mathbf{x}_k) + \nabla^2 f(\mathbf{x}_k) x = 0. \tag{1.3}$$

Namely, the pure Newton direction p_k is the solution of the system

$$\nabla^2 f(\mathbf{x}_k) p_k = -\nabla f(\mathbf{x}_k), \tag{1.4}$$

and the next Newton iterate is

$$\mathbf{x}_{k+1} = \mathbf{x}_k + p_k. \tag{1.5}$$

When $\nabla^2 f(\mathbf{x}_k)$ is non-singular, the closed-form expression for the Newton direction is

$$p_k = -\left[\nabla^2 f(\mathbf{x}_k)\right]^{-1} \nabla f(\mathbf{x}_k). \tag{1.6}$$

Moreover, when $\nabla^2 f(\mathbf{x}_k)$ is positive definite, p_k is a descent direction, since the directional derivative at $\mathbf{x}_k$ in the direction of p_k is

$$\nabla f(\mathbf{x}_k)^T p_k = -\nabla f(\mathbf{x}_k)^T \left[\nabla^2 f(\mathbf{x}_k)\right]^{-1} \nabla f(\mathbf{x}_k) < 0. \tag{1.7}$$

Under certain conditions, Newton's method enjoys a local quadratic convergence property that cannot be matched by first-order methods. Hence, when the second derivatives of f can be computed, and f is strictly convex, Newton's method is viable, and quite powerful considering that the gradient and Hessian can be computed efficiently up to arbitrary precision via automatic differentiation.

Nevertheless, as hinted above, the pure Newton method has multiple conspicuous failure-modes. The first problem is that pure Newton is not globally convergent even when the Hessian is positive definite at each point. Luckily, this can be fixed with a simple backtracking line search routine that chooses a step-size based on the Armijo sufficient decrease condition. This backtracking line search routine will be applied to all the methods throughout the project.

More problems arise when f is not strictly convex, i.e., when the Hessian is not

positive definite or ill-conditioned. When the Hessian has a negative eigenvalue, the pure Newton method may move up towards a local maximum, or a saddle point. When the Hessian has a zero eigenvalue, or is severely ill-conditioned, the next pure Newton iterate cannot even be calculated, or is highly inaccurate. Modified Newton methods that address these issues are introduced in section 2.

Another issue that often arises in data science is that evaluating the gradient and the Hessian matrix is expensive for a function that is a large sum of component functions

$$f(x) = \frac{1}{n}\sum_{i=1}^{n} f_i(x). \tag{1.8}$$

These types of functions arise frequently in data science, where the components $f_i(x)$ are often loss functions corresponding to some metric. For experiments with a huge number of observations, namely when n is large, it may be prohibitively expensive to compute the exact Hessian for each Newton iteration. Randomized methods known as matrix sketching aim to form an approximate Hessian that retains second order information, and is easier to compute than the exact Hessian. Moreover, when d is large, Solving 1.4 using traditional deterministic techniques require an $\mathcal{O}(d^3)$ operation, and it may even be infeasible to store the whole $d \times d$ Hessian in core memory. In this case, techniques from randomized linear algebra allow one to solve a massive linear system either directly [6, 5], or using an iterative method [10, 4]. Such methods are discussed in section 3.

2. Modified Newton techniques

2.1. Dealing with non-convex functions or ill-conditioned Hessian

When the Hessian $\nabla^2 f(\mathbf{x}_k)$ is not positive definite, the inequality (1.7) is no longer valid, hence p_k may not even be a descent direction. Moreover, if $\nabla^2 f(\mathbf{x}_k)$ is singular, a direction p, which is the solution to the system (1.4), may not even exist. Even in these cases, as long as the leading eigenvalue of the Hessian is sufficiently positive, one can find a direction that uses as much curvature information as possible.

One obvious way to always get a descent direction at each iteration is to perform gradient descent when the Hessian is not positive definite, and use the Newton direction when the Hessian is positive definite. However, this is not much better than solely performing gradient descent when there are not many points where the function is strictly convex. Furthermore, the calculations used to determine whether the Hessian is positive definite at each step are largely wasted.

Alternatively, one can replace the true Hessian H with an approximate Hessian $\hat{H}$ that is positive definite, and whose condition number is bounded by a certain number. The most natural method is to take the spectral decomposition of H,

$$H = UDU^T, \tag{2.1}$$

and replace the eigenvalues that are smaller than a certain threshold with a positive number. More specifically, let λ_1 denote the largest eigenvalue of H, and $\kappa_{\max}$ the desired condition number of $\hat{H}$. Then all the eigenvalues of H that are smaller than

$\lambda_1/\kappa_{\max}$ are causing the condition number of H to be larger than $\kappa_{\max}$. Therefore, one could define $\hat{H}$ as

$$\hat{H} = U\hat{D}U^T, \tag{2.2}$$

where $\hat{D}$ is the diagonal matrix which contains all the eigenvalues of H that are greater than $r := \lambda_1/\kappa_{\max}$, and has r in the diagonal entries where the original eigenvalue is smaller then r. Defining $\hat{H}$ this way is particularly useful because the distance between H and $\hat{H}$ in the operator two-norm is bounded by $r - \lambda_n$, where λ_n is the smallest eigenvalue of H. Assuming that the eigenvalues are in descending order, this fact is directly observable by the following:

$$\begin{aligned}
\|H - \hat{H}\|_2 &= \|UDU^T - U\hat{D}U^T\|_2 \\
&= \|U(D - \hat{D})U^T\|_2 \\
&= \left\| U \begin{pmatrix} 0 & & & & & \\ & \ddots & & & & \\ & & 0 & & & \\ & & & r-\lambda_k & & \\ & & & & \ddots & \\ & & & & & r-\lambda_n \end{pmatrix} U^T \right\|_2 \\
&= r - \lambda_n.
\end{aligned} \tag{2.3}$$

Alternatively, instead of replacing all eigenvalues smaller than r to be r, one can find another positive definite approximation $\tilde{H}$ that switches the sign of the negative eigenvalues, and replaces the eigenvalues with absolute value less than r to be r. By a similar procedure as in equation (2.3), the distance between $\tilde{H}$ and H in this case is

$$\|H - \tilde{H}\|_2 = \begin{cases} -2\lambda_n & (|\lambda_n| \geq r) \\ r - \lambda_n & (|\lambda_n| < r) \end{cases}. \tag{2.4}$$

Hence, speaking only in terms of linear algebra, $\hat{H}$ is closer to H than $\tilde{H}$ is when the smallest negative eigenvalue of H is large in absolute value. However, this does not necessarily mean that $\hat{H}$ produces a better step in terms of optimization, and decisions should be made on a case-by-case basis.

Methods of making modifications such as (2.2) for large systems are discussed in section 3.

2.2. Numerical experiment: ordinary least squares regression

The modified Newton method (2.2) is used to perform an ordinary least squares regression on a synthetic data matrix $X \in \mathbb{R}^{n\times(d+1)}$ with d features and n observations. The first column of X is a column of ones, which corresponds to the constant term. This dataset is constructed by drawing d random numbers from a uniform(-1,1) distribution for every observation. The output vector $\mathbf{y}$ is constructed by the following rule

$$\mathbf{y} = X\beta^* + \varepsilon, \tag{2.5}$$

where $\beta^* = [\beta_0^*, \ldots, \beta_d^*]^T$ is determined arbitrarily, and $\varepsilon_i \sim \mathcal{N}(0, 10^{-4})$ for all $i = 1, \ldots, n$.

There are many ways to find least squares solution $\hat{\beta}$, for instance, via the normal equation, the QR factorization, or the SVD, etc.

The least squares solution can also be found by solving the minimization problem

$$\min_{\beta \in \mathbb{R}^{d+1}} \frac{1}{2} \|\mathbf{y} - X\beta\|_2^2 = \frac{1}{2} \sum_{i=1}^{n} \left(y_i - \sum_{j=1}^{d+1} X_{ij}\beta_j \right)^2 . \tag{2.6}$$

The objective function $\|\mathbf{y} - X\beta\|^2$ is often called the residual sum of squares (RSS), and its gradient is

$$g := \nabla_\beta \mathrm{RSS} = \left[-\sum_{i=1}^{n} \left(y_i - \sum_{j=1}^{d+1} X_{ij}\beta_j \right) X_{ij} \right]_j , \tag{2.7}$$

and the Hessian is

$$H := \nabla_\beta^2 \mathrm{RSS} = \left[\sum_{i=1}^{n} x_{ik} x_{ij} \right]_{jk} . \tag{2.8}$$

The RSS is a convex quadratic function, hence H will not have any negative eigenvalues. Therefore, the saddle point problem is not an issue in OLS regression. However, one can purposefully cause the Hessian to be ill-conditioned by making some features in the data matrix have strong correlation. Namely, if two columns of X are an exact scalar multiple of one another, the hessian H will have a zero eigenvalue, causing it to be singular. Moreover, one can control the condition number of H by generating two correlated columns

$$(X_{ki})_k = (X_{kj} + \xi_k)_k , \tag{2.9}$$

where $\xi \sim \mathcal{N}(0, \sigma^2)$. The smaller σ is, the greater the two columns are correlated, hence the greater the condition number of H.

The figures 1, 2, 3 are the plots for $d = 1,\ n = 10^3,\ \beta^* = [1, 1]^T$. The Hessian H was made increasingly ill-conditioned until the pure Newton method started to malfunction.

The interesting thing about the modified Newton method ("replace"), is that when the two columns of X are strongly correlated, the iterates do not converge to the underlying β^* that was used to generate $\mathbf{y}$. Instead, the modified Newton method converges to a different solution $\hat{\beta}$, but the sum $\hat{\beta}_0 + \hat{\beta}_1$ approximately retrieves $\beta_0^* + \beta_1^*$.

A similar pattern is observed for $d = 20,\ n = 10^3,\ \beta^* = \left[1, 1, \frac{1}{10}, \ldots, \frac{1}{10}\right]^T$. The first and second columns of X are generated according to

$$(X_{i1})_i = (X_{i2}) + \xi_i, \tag{2.10}$$

where $\xi_i \sim \mathcal{N}(0, (10^5)^2)$.

Figure 4 shows the coordinates of the modified Newton and pure Newton iterates after 20 iterations compared to the coordinates of the least squares solution obtained

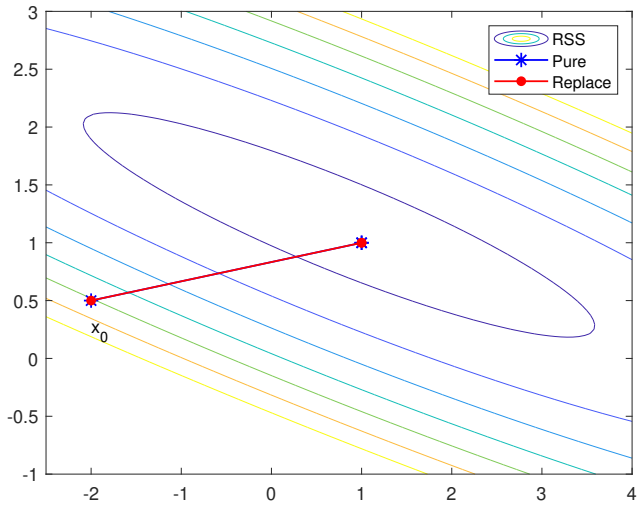

Figure 1: $\xi_i \sim \mathcal{N}(0, 0.16)$. H is well conditioned so that both pure Newton ("pure"), and modified newton ("replace") converge in one iteration.

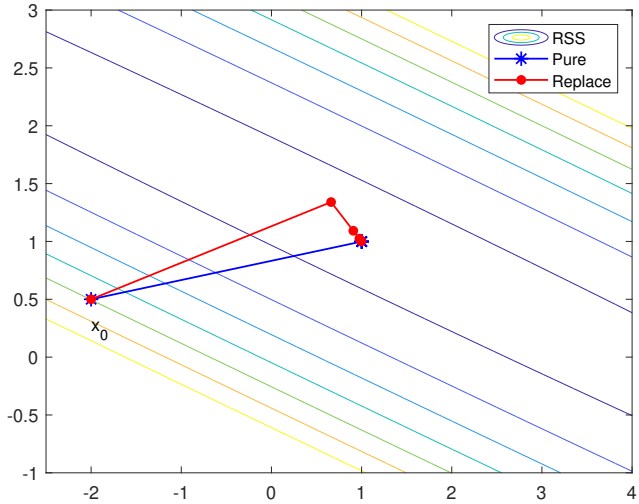

Figure 2: $\xi_i \sim \mathcal{N}(0, 0.0025)$. The eigenvalues of H have become so small that $\hat{H} \neq H$.

via the QR factorization of X. In figure 4, $\hat{\beta}_R$ is the iterate of the modified Newton method, $\hat{\beta}_P$ is an iterate of the pure Newton method, and $\hat{\beta}_{LS}$ is the least squares solution obtained via the QR factorization.

It is interesting that the difference between the pure Newton iterate and the least square solution obtained via QR is very similar. The first and second coordinates of β_P, which correspond to the highly correlated features in X add up to approximately $\beta_0^* + \beta_1^* = 2$, but the first two coordinates soon diverge. The least squares solution via QR also suffers from the ill-conditioning of the matrix $X^T X$. The modified Newton coordinates converge to the specific limit in figure 4 even when the Hessian is highly ill-conditioned.

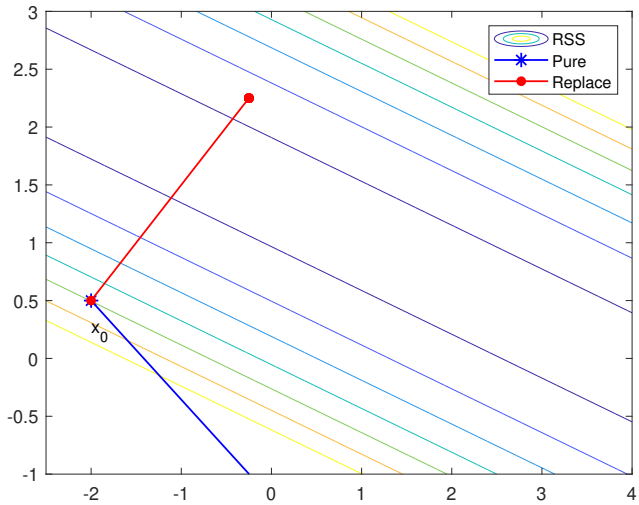

Figure 3: $\xi_i \sim \mathcal{N}(0, 10^{-8})$. The condition number of H is so large that pure Newton diverges. Modified Newton converges to a point where $\beta_0^* + \beta_1^* \approx \hat{\beta}_0 + \hat{\beta}_1$.

Although these experiments were conducted on the convex quadratic RSS, which is somewhat uninteresting, this method can be directly applied to other loss functions such as the log loss function, which appears in logistic regression.

3. Matrix sketching techniques

In the previous sections, the data matrix $X \in \mathbb{R}^{n \times d}$ only had a small number of observations ($n \leq 10^3$). When n becomes large, it may be difficult to form and evaluate the Hessian matrix directly. Instead, a stochastic method can be used to find an approximate hessian rather than computing the exact Hessian at each iteration. In particular, in the case that the Hessian matrix $\nabla^2 f$ can be decomposed into

$$\nabla^2 f = R^T R, \tag{3.1}$$

it is shown in [8] that by choosing a suitable sketch matrix $S \in \mathbb{R}^{m \times n}$ that satisfies $\mathbb{E}[S^T S/n] = I_n$, the approximate Hessian

$$A := (SR)^T SR \tag{3.2}$$

yields desirable results.

The RSS in OLS regression (2.6), which is the objective function that is used in this project, is an example where a decomposition of the Hessian (3.1) is easy to compute. Indeed, by observing the expression of the Hessian of the RSS in (2.8), it is clear that the Hessian can be written as

$$\nabla^2 \text{RSS} = \left(\sum_{i=1}^{n} x_{ij} x_{ik} \right)_{jk} = X^T X, \tag{3.3}$$

$\hat{\beta}_R$	$\hat{\beta}_P$	$\hat{\beta}_{LS}$
-2.497405e-01	-6.033845e+01	-6.033845e+01
2.250259e+00	6.233896e+01	6.233896e+01
9.973904e-02	9.973376e-02	9.973376e-02
1.008449e-01	1.008807e-01	1.008807e-01
1.009253e-01	1.009292e-01	1.009292e-01
9.996128e-02	9.998783e-02	9.998783e-02
9.932395e-02	9.935729e-02	9.935729e-02
1.001781e-01	1.002072e-01	1.002072e-01
9.981634e-02	9.986353e-02	9.986353e-02
1.011287e-01	1.011002e-01	1.011002e-01
1.004734e-01	1.004555e-01	1.004555e-01
9.948072e-02	9.944194e-02	9.944194e-02
9.973338e-02	9.973072e-02	9.973072e-02
1.000790e-01	1.000892e-01	1.000892e-01
9.979954e-02	9.982520e-02	9.982520e-02
1.002446e-01	1.001961e-01	1.001961e-01
9.902866e-02	9.903539e-02	9.903539e-02
9.992277e-02	9.988797e-02	9.988797e-02
9.964168e-02	9.965509e-02	9.965509e-02
9.933245e-02	9.936934e-02	9.936934e-02
$\|\hat{\beta}_P - \hat{\beta}_{LS}\|_2 = 4.5421$e-09		

Figure 4: $\hat{\beta}_R$ is the 10-th iterate of modified Newton, $\hat{\beta}_P$ is the 10-th iterate of pure Newton, and $\hat{\beta}_{LS}$ is the least squares solution via QR. The iterates from pure Newton are close to the least squares solution obtained via QR.

where $X \in \mathbb{R}^{n\times d}$ is simply the data matrix, and x_{ij} is the i, j-th entry of X.

In this section, we discuss randomized techniques known as matrix sketching, that allows one to compute an approximate Hessian of a large sum of loss functions. When there is also a large number of variables or features, namely when d is large, such sketching techniques are still viable, but one must also pair it with a large linear system solver as in [1]. Due to the symmetric positive semi-definiteness of the expression (3.2), a conjugate gradient solver is effective, and can be implemented so that only matrix-vector products are required, and the approximate Hessian need not be fully formed.

3.1. Overview of methods

The two methods in [8] are implemented in this project. As a benchmark, a simple stochastic gradient descent is also implemented.

i. Newton Sketch

As discussed briefly, the goal is to find a sketching matrix S such that $\mathbb{E}[S^T S/n] = I_n$, and (3.2) is easy to compute. A direct computation of the Hessian via equation (3.3) would cost $O(nd^2)$ flops. However, if the matrix-matrix multiplication SX can be performed quickly, then $(SX)^T SX$ requires less work.

A Gaussian sketch matrix, namely a matrix S whose entries are all realizations of a standard Normal distribution, has well-understood convergence properties, but

does not facilitate the computation of (3.2). Alternatively, let

$$S = \sqrt{n}EHD, \tag{3.4}$$

where the rows of $E \in \mathbb{R}^{m\times n}$ are standard unit basis vectors e_j of $\mathbb{R}^n$, where j is chosen uniformly from $[\![1, \dots n]\!]$ with replacement, H is the Hadamard matrix of size n scaled by $1/\sqrt{n}$, and D is a diagonal matrix whose diagonal consists of Rademacher random variables. E and D are very sparse, so applying these matrices only require at most $O(n)$ flops. The Hadamard matrix H can be applied in $O(n \log n)$ flops via a fast Walsh-Hadamard transform. Thus, SX can be computed in near-linear time in total. In the matlab code `NewtonSketch.m`, the matrix H is applied via a simple divide-and-conquer algorithm using the recursive property

$$H_n Y = \left[\frac{H_{n-1}Y_t + H_{n-1}Y_b}{H_{n-1}Y_t - H_{n-1}Y_b} \right], \tag{3.5}$$

where H_n is the Hadamard matrix of size n, and Y_t, Y_b are the equally sized top and bottom halves of Y respectively. When $A_k := (SX)^T SX$ is invertible, the Newton step is given as $p_k = -A_k^{-1}\nabla f_k$.

ii. Row-sampled Newton

The row-sampled Newton, which is a simple case of a class of sub-sampled Newton methods introduced in [9], is similar in spirit to the stochastic gradient descent. In order to minimize a big sum of loss functions, the random row-sampling selects a batch of functions within the sum with replacement, and computes an approximate Hessian only using the selected functions in each iteration. In other words, a fixed number of rows in the data matrix X are chosen to compute an approximate Hessian. Then, the Newton step is computed by solving the system of equations

$$\widetilde{H}p = -\nabla f. \tag{3.6}$$

iii. Stochastic Gradient Descent

In comparison to ii, the vanilla stochastic gradient descent selects a batch of functions with replacement within the sum, and computes an approximate gradient, then the next iterate is simply the approximate gradient subtracted from the current iterate. Namely,

$$x_{k+1} = x_k - \widetilde{\nabla f}. \tag{3.7}$$

There is a vast literature on many variants of the stochastic gradient descent, but the vanilla version with batch size $m = 1000$ is implemented in this project.

3.2. Numerical experiments

In [8], the termination criteria for the Newton algorithm is based on the quantity

$$\lambda_f(x_k) := \nabla f(x_k)^T \Delta x_k, \tag{3.8}$$

which is called the optimality gap. Four types of plots are created as a result of the experiments in this section: iterations vs. optimality gap, iterations vs. $\|\nabla f\|_2$, n vs. required iterations, and finally n vs. run-time.

For all methods, the number of features were kept at $d = 100$. For Newton Sketch, the sketch dimension m was chosen to be $1.2d$, which yielded the best results for this particular experiment. For both random row-sampling and SGD, the batch size was chosen to be 1000.

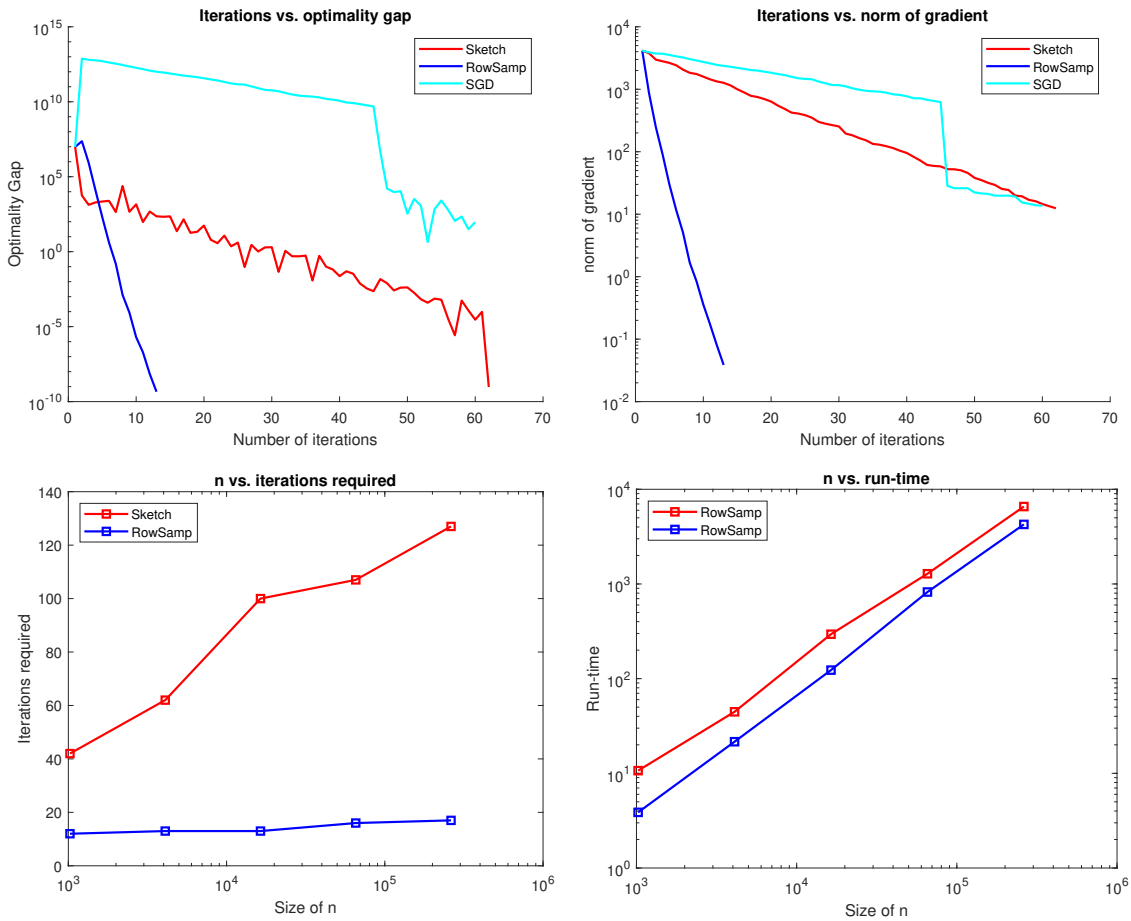

Figure 5: The top two plots show the convergence of each method at $n = 2^{12}$. It can be seen that random row-sampled Newton performs very well on this data set. SGD makes a big descent around iteration 50, but starts to flounder after that. The norm of the gradient at iteration 50 is similar for SGD and Newton Sketch, but Newton Sketch converges steadily afterwards, while SGD flounders.

3.3. Remarks

Due to the nature of the synthetic data set, the experiments in the previous section show that random row-sampling displays better performance than Newton Sketch. However, it is demonstrated in [1] that Newton Sketch is highly effective for other data sets and different linear models.

It is also worth noting that when the hessian is separable in the sense of (3.1), the Hessian, or the approximate hessian computed via (3.2) does not have negative

n	Newton Sketch	Random row-sampling
1024	1.0677e+01	3.8697e+00
4096	4.4647e+01	2.1572e+01
16384	2.9467e+02	1.2305e+02
65536	1.2819e+03	8.2219e+02
262144	6.5713e+03	4.2569e+03

Figure 6: Time stamps for each method

eigenvalues, hence f will not have any saddle points. However, when the data matrix X is column rank-deficient, the exact Hessian matrix and approximate Hessian matrices obtained via the Newton Sketch or random row-sampling may become singular or ill-conditioned. In this case, one must modify some of the smaller eigenvalues of the approximate Hessian by first computing an eigenvalue-revealing decomposition such as (2.1). Even when d is large, and the full Hessian cannot be stored in core memory, there are standard randomized techniques such as in [5, 6, 12] that allows one to compute an approximate matrix factorization. Effectively updating smaller eigenvalues of large Hessian matrices is a topic to be further investigated.

4. Conclusion

It was seen that simple modifications of Newton's method can fix the intrinsic failure modes of pure Newton's method. Global convergence for convex functions was achieved by adding a line-search routine satisfying the Armijo or Woolfe sufficient decrease conditions. The saddle point problem, and the problem with ill-conditioned Hessian matrices can be fixed by replacing some of the badly behaving eigenvalues of the Hessian.

Furthermore, in applications in statistics, it is shown that for a data matrix $X \in \mathbb{R}^{n\times d}$, Newton's method can be scaled to handle larger values of n using matrix sketching techniques. Larger d can also be dealt with using an iterative solver such as the conjugate gradient method. As mentioned in 3.3, further investigation is required for the case where the approximate Hessian is singular or ill-conditioned.

References

[1] A. S. Berahas, R. Bollapragada, and J. Nocedal. "An Investigation of Newton-Sketch and Subsampled Newton Methods". In: *Optimization Methods and Software* (2020). DOI: 10.1080/10556788.2020.1725751. URL: https://arxiv.org/abs/1705.06211.

[2] J. E. Dennis and R. B. Schnabel. *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*. SIAM, 1996. ISBN: 978-0-89871-364-0. DOI: https://doi.org/10.1137/1.9781611971200.

[3] M. Derezinski et al. "Newton-LESS: Sparsification without Trade-offs for the Sketched Newton Update". In: Neural Information Processing Systems, 2021.

[4] R. M. Gower and P. Richtárik. "Randomized Iterative Methods for Linear Systems". In: *SIAM J. Matrix Anal. Appl* 36.4 (2015), pp. 1660–1690. DOI: https://doi.org/10.1137/15M1025487. URL: https://arxiv.org/abs/1506.03296.

[5] N. Halko, P. G. Martinsson, and J. A. Tropp. "Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions". In: *SIAM Review* 53.2 (2011), pp. 217–288. URL: https://arxiv.org/abs/0909.4061.

[6] P. G. Martinsson. "Randomized methods for matrix computations". In: *IAS/-Park City Mathematics Series* (2018). URL: https://arxiv.org/abs/1607.01649.

[7] J. Nocedal and S. J. Wright. *Numerical Optimization*. Springer, 2006.

[8] M. Pilanci and M. J. Wainwright. "Newton Sketch: A Linear-time Optimization Algorithm with Linear-Quadratic Convergence". In: *SIAM J. Optim.* 27.1 (2017), pp. 205–245. DOI: https://doi.org/10.1137/15M1021106. URL: https://arxiv.org/abs/1505.02250.

[9] F. Roosta-Khorasani1 and M. W. Mahoney. "Sub-sampled Newton methods". In: *Mathematical Programming* 174 (2019), pp. 293–326. DOI: https://doi.org/10.1007/s10107-018-1346-5.

[10] T. Strohmer and R. Vershynin. "A Randomized Kaczmarz Algorithm with Exponential Convergence". In: *Journal of Fourier Analysis and Applications* 15.262 (2009). DOI: https://doi.org/10.1007/s00041-008-9030-4. URL: https://arxiv.org/abs/math/0702226.

[11] J. A. Tropp et al. "Practical Sketching Algorithms for Low-Rank Matrix Approximation". In: *SIAM J. Matrix Anal. Appl* 38.4 (2017), pp. 1454–1485. DOI: https://doi.org/10.1137/17M1111590.

[12] F. Woolfe et al. "A fast randomized algorithm for the approximation of matrices". In: *Applied and Computational Harmonic Analysis* 25.3 (2008), pp. 335–366. ISSN: 1063-5203. DOI: https://doi.org/10.1016/j.acha.2007.12.002. URL: https://www.sciencedirect.com/science/article/pii/S1063520307001364.